

第十一章 外部排序

引言

11.1 外存信息的存取

11.2 外部排序的基本方法—归并排序法

11.3 多路平衡归并

11.4 置换-选择排序

11.5 最佳归并树

本章学习要点、习题

[外部排序的应用对象]

保存在外存储器上的信息量很大（**远远大于内存可存储的记录数量**）的数据记录文件。

[外排序面临的问题]

无法将所有记录一次性读入内存，无法采用内排序一次性将排序完成；结合分批读入内存的处理方法想办法。

[外排序与内排序的差别]

- 内部排序充分利用内存可以随机存取的特点，如
 - 希尔排序中，相隔 di 的记录关键字可作比较；
 - 堆排序中，完全二叉树中的父 $R[i]$ 与子 $R[2i]$ 、 $R[2i+1]$ 可比；
 - 快速排序中，需正向和逆向访问记录序列
- 外存信息的定位和存取受其物理特性的限制

[外部排序的实现手段]

在排序过程中，进行多次内外存之间的数据交换

11.1 外部信息的存取

11.1.1 磁带信息的存取

属于**顺序存取设备**，只适用于存储顺序文件

[工作原理]

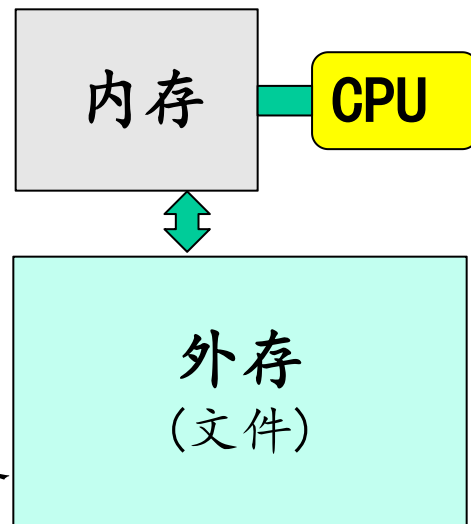
磁带不是连续运转设备，而是启停设备
I/O操作以块为单位



读写一块信息的时间 $T_{I/O} = t_a + nt_w$

t_a : 用于磁头定位在物理块起始位置的时间

t_w : 传输一个字符的时间



11.1.2 磁盘信息的存取

属于**直接存取设备**，适用于各类存储形式的文件
[工作原理]

磁盘可以是单片的，或有若干个盘片组成盘组

磁道：盘片上的同心圆

柱面：各盘面上半径相同的磁道合在一起

扇区：每个磁道等分为若干扇区(块)

读取信息的过程：

柱面号 移动磁头

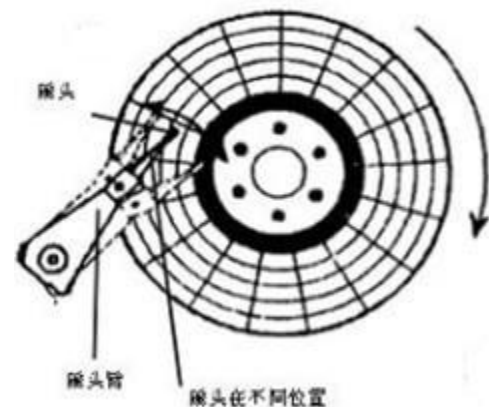
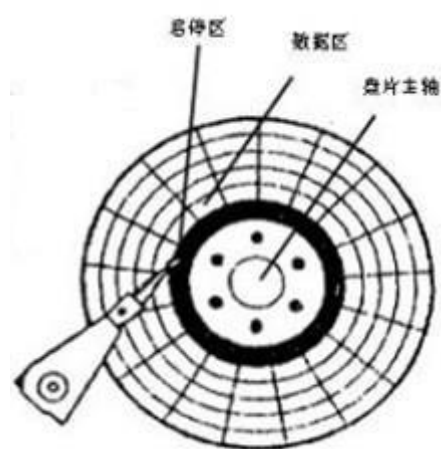
盘面号(磁道号) 选定所用磁头

扇区/块号 磁盘一直高速旋转，磁头等待开始位置

读写一块信息的时间 $T_{I/O} = t_{seek} + t_{la} + nt_w$

t_{seek} ：磁头定位到磁道的时间()

t_{la} ：等待时间

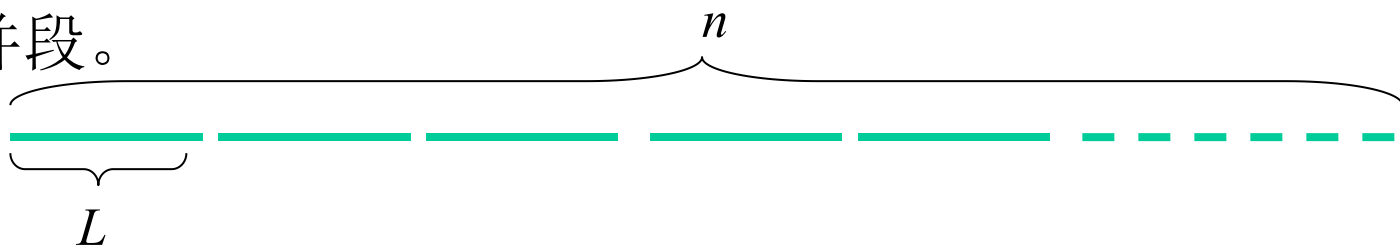


11.2外部排序的基本方法—归并排序法

[步骤]

1) 生成若干初始归并段(初始归并串/顺串)，即对文件进行预处理

- 把含有 n 个记录的文件，按内存缓冲区大小分成若干长度为 L 的可一次完整装入内存的子文件/段；
- 将每个段，依次调入内存用有效的内排序方法排序后送回外存，形成若干个“初始归并段”；
- 如下图： n 个记录，每 L 个记录进行内部排序形成一个初始归并段。



2) 对初始归并段进行多路合并(多路归并)

初始方法：对初始归并段进行多次2路归并排序，直至最后在外存上得到整个有序文件为止（原理同内排序的2路归并排序）

例如：假设有一个含10,000个记录的磁盘文件，而当前所用的计算机一次只能对1,000个记录进行内部排序：

- 1、则首先利用内部排序的方法得到10个 $(10,000/1,000)$ 初始归并段，然后进行逐趟归并。
- 2、先进行2-路归并(即两两归并)，则第一趟由10个归并段得到5个归并段（ $5*2000$ 条）；
- 3、第二趟由 5 个归并段得到3个归并段（ $2*4000+1*2000$ 条）；
- 4、第三趟由 3 个归并段得到2个归并段（ $6000+4000$ 条 或 $8000+2000$ 条）；
- 5、最后一趟归并得到整个记录的有序序列（10000条）。

假设“数据块”的大小为200，即每一次访问外存可以读/写200个记录，则对于10,000个记录，全部处理一遍需访问外存100次(读和写各50次)；分析上述外排过程中访问外存(对外存进行读/写)的次数：

- ◆ 1) 求得10个初始归并段需访问外存100次；
- ◆ 2) 第一趟和第四趟进行了全部记录的归并；需访问外存 2×100 次；
- ◆ 3) 第二趟未进行全部记录的归并；因本趟有一段2000条可以不参加归并， $1 \times 100 - 10 \times 2 = 80$ 次
- ◆ 4) 第三趟未进行全部记录的归并；按条数多未一段4000条未参加来计算，访问外存 $1 \times 100 - 10 \times 4 = 60$ 次；
- ◆ 5) 总计访问外存 $100 + 200 + 80 + 60 = 440$ 次

外排总的时间还应包括内部排序所需时间和逐趟归并时进行内部归并的时间

$$\begin{aligned}\text{外部排序总时间} = & \text{产生初始归并段的时间 } m * t_{IS} \\ & + \text{外存信息读写时间 } d * t_{IO} \\ & + \text{内部归并所需时间 } s * ut_{mg}\end{aligned}$$

m : 初始归并段个数; t_{IS} : 一次内部排序耗时;
 d : 归并排序的全部读写次数; t_{IO} : 单次磁盘读写的磁盘IO时间;
 s : 归并趟数; ut_{mg} : u 个记录内存归并排序的时间

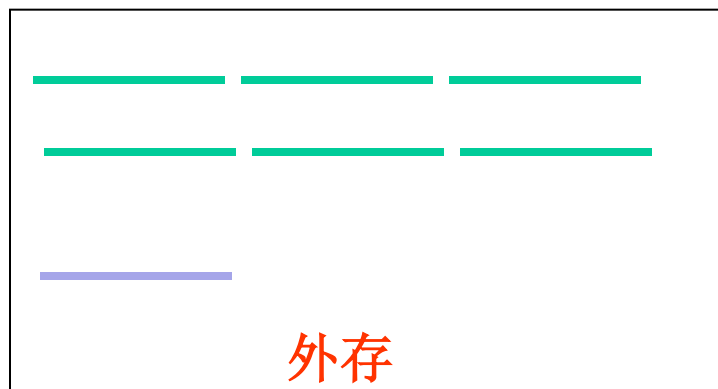
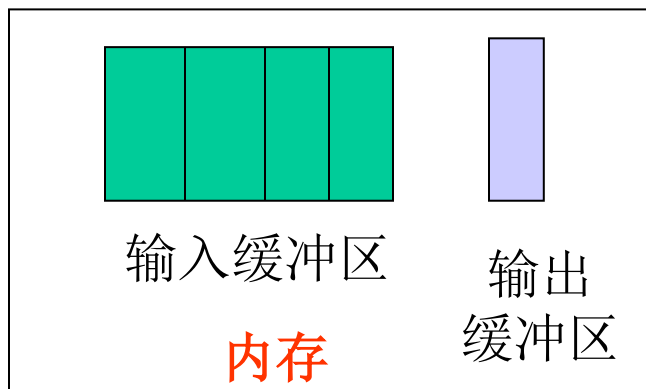
t_{IO} 值是外存操作时间，远远大于 t_{IS} 和 t_{mg} 。

外部排序总时间，即取决于读写外存的次数 d 。

[提高外排序效率(缩短总时间)的途径]

多路归并时是树型结构将多个初始归并段合并成为一个已排序段，因此总合并次数 $s = \lceil \log_k m \rceil$ ，优化目标：减少合并趟数 s ，以减少I/O次数

- 方向1，减少 m ：在内存受限的情况下，如何扩大初始归并段长度（大于内存容纳数量），从而减少初始归并段个数 m ，采用**置换-选择排序方法建立初始归并段**；
- 方向2，增加 k ：进行**基于败者树的多路(k 路， $k > 2$)归并**。由于内存中可以容纳多个记录，归并时每个段每次最少只需读入1次I/O可读入的少量记录即可：设内存可容纳5次I/O的物理块，则可将4个作为输入缓冲区，一个作为输出缓冲区进行4路归并



一般情况下，假设待排记录序列含 m 个初始归并段，外排时采用 k -路归并，则归并趟数 $s = \lceil \log_k m \rceil$ ，显然，随着 k 的增大或 m 的减小，归并的趟数将减少，因此对外排而言，通常采用多路归并。 k 的大小可选，但需综合考虑各种因素。

例如：

若对上述例子采用 2-路归并，要进行 4 趟归并，外排所需的总的访问外存次数：440 次

若对上述例子采用 5-路归并，则只需进行 2 趟归并，外排所需总的访问外存的次数为 $100 + 2 \times 100 = 300$ 次

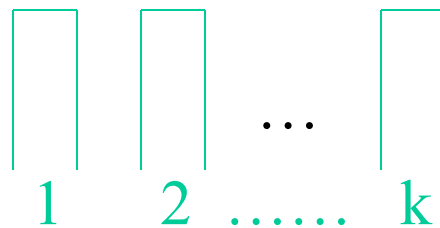
11.3 多路(k路)平衡归并

[采用简单比较方法（简单选择排序）进行多路归并存在的问题]

设，记录总数为 n ，初始归并段的个数为 m ，进行 k 路归并合并，共进行 $s = \lceil \log_k m \rceil$ 趟归并；归并时采用简单选择排序从 k 个记录中选择一个关键字最小的记录时的比较次数为 c （简单选择，即为 $k-1$ ），则 s 趟内部归并总的比较次数为：

$$C_{\text{总}} = s \times c \times (n-1) = \lceil \log_k m \rceil \times c \times (n-1) = \lceil \log_2 m / \log_2 k \rceil \times c \times (n-1)$$

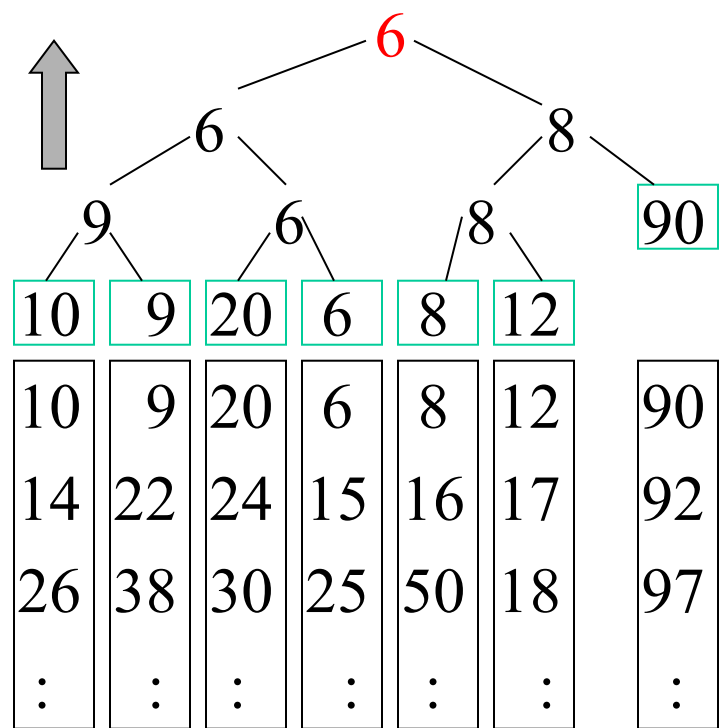
矛盾 $\begin{cases} k \uparrow, s \downarrow, \text{可减少I/O次数;} \\ k \uparrow, c = (k-1) \uparrow, \text{归并效率} \downarrow \end{cases}$



[解决矛盾的方法]

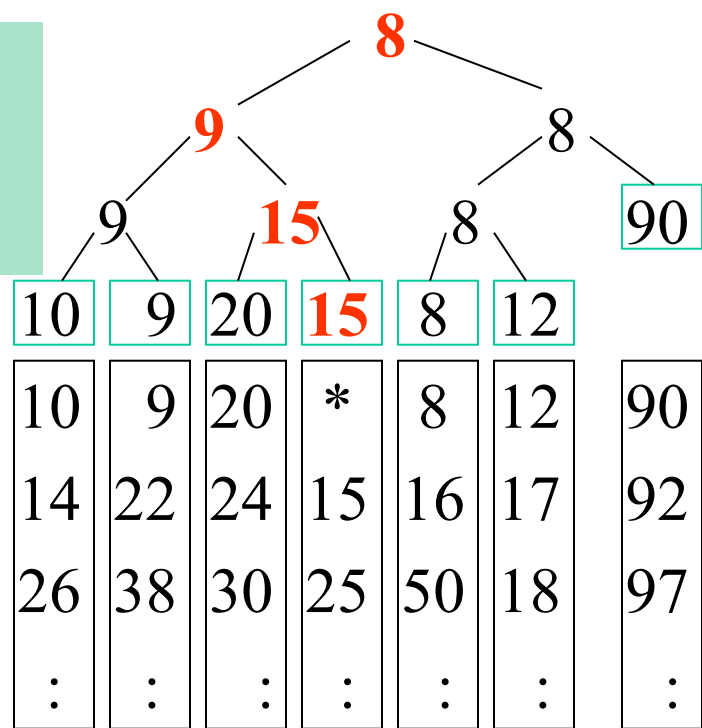
——对 k 个数里选最小值的简单选择排序方法进行优化：采用树形选择排序（胜者树、败者树方法），选关键字最小的记录，此时 $c = \log_2 k$ ，则 $C_{\text{总}} = \lceil \log_2 m \rceil (n-1)$ ，与 k 无关

[胜者树(树形选择排序)]



7路合并胜者树

输出缓冲区



输出缓冲区

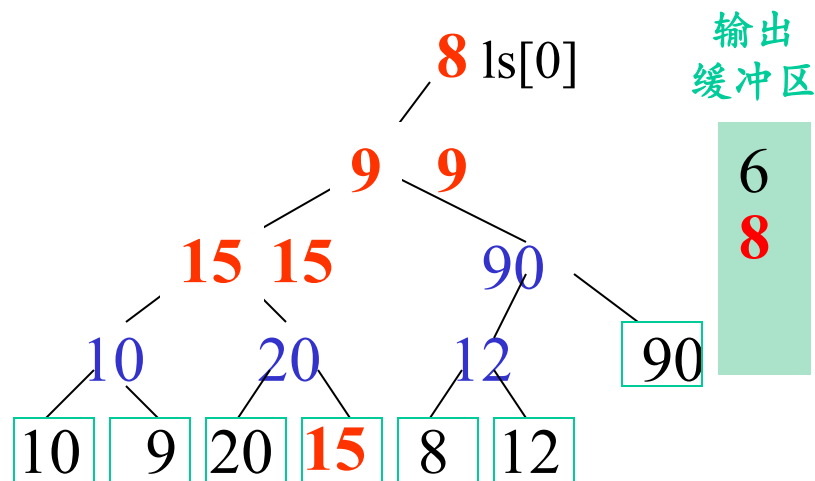
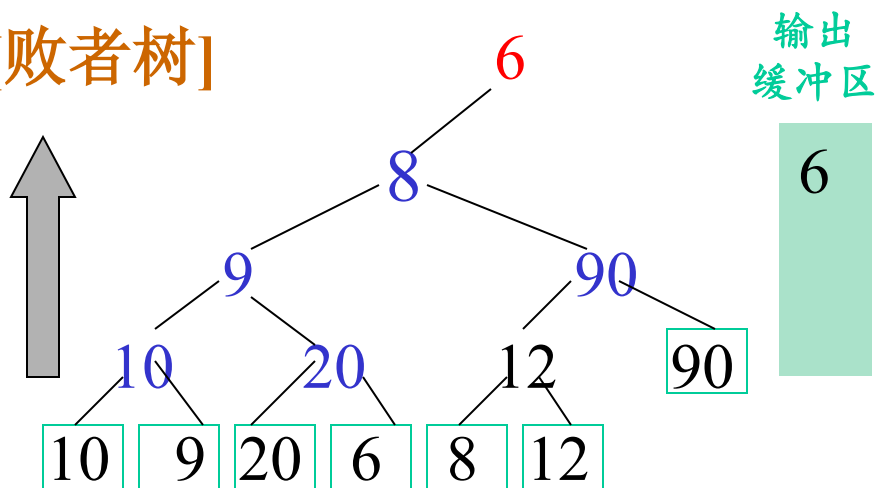


7路归并时胜者树的重构

树形选择排序，称为胜者树，即树中保留两两比较的胜者。

重构胜者树时，从胜者所在的归并段中补充一个新数据到胜者原叶子结点，再从此叶子结点向根方向依次比较更新一遍。这样从根结点至新补入记录的叶结点的路径上的所有结点都必须更新。

[败者树]



10	9	20	6	8	12	90
14	22	24	15	16	11	92
26	38	30	25	50	18	97
:	:	:	:	:	:	:

7路合并败者树

10	9	20	*	8	12	90
14	22	24	15	16	11	92
26	38	30	25	50	18	97
:	:	:	:	:	:	:

重构后的败者树

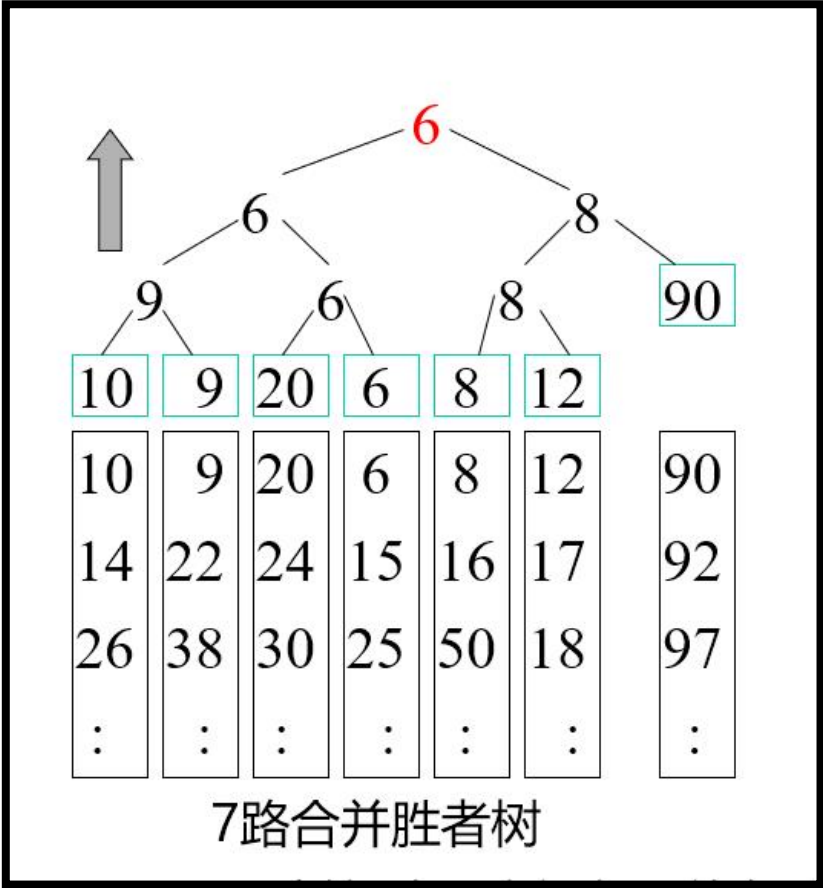
败者树的特点：记录败者，胜者参加下一轮比赛

败者树的优点：重构时修改结点较少。

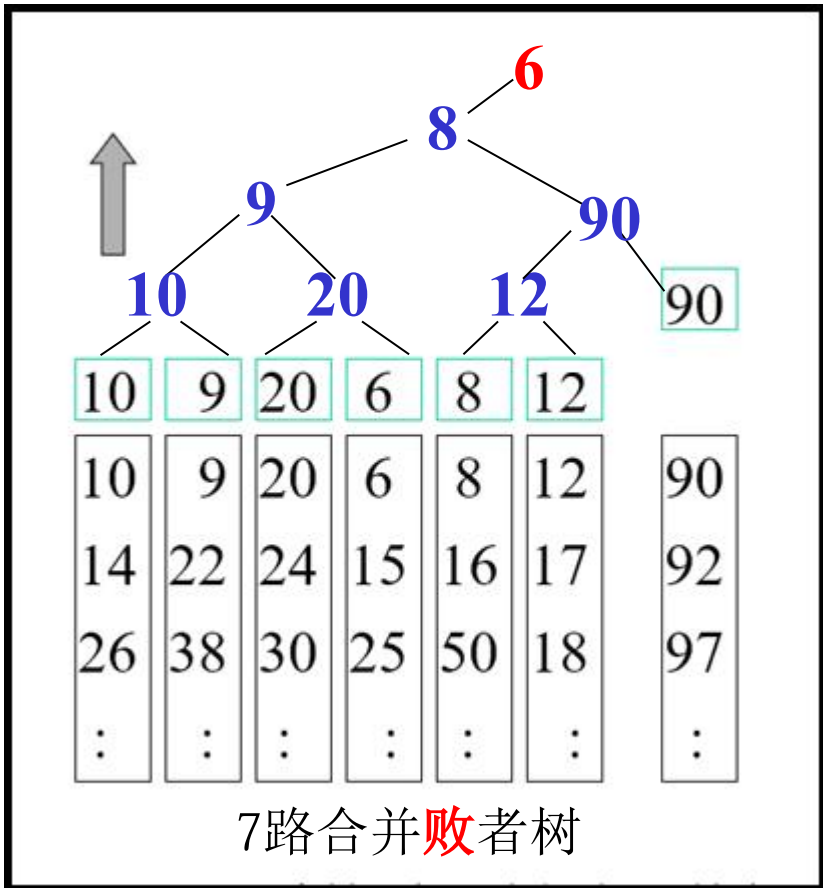
[胜者树(树形选择排序) 和 败者树对比]

输出
缓冲区

6

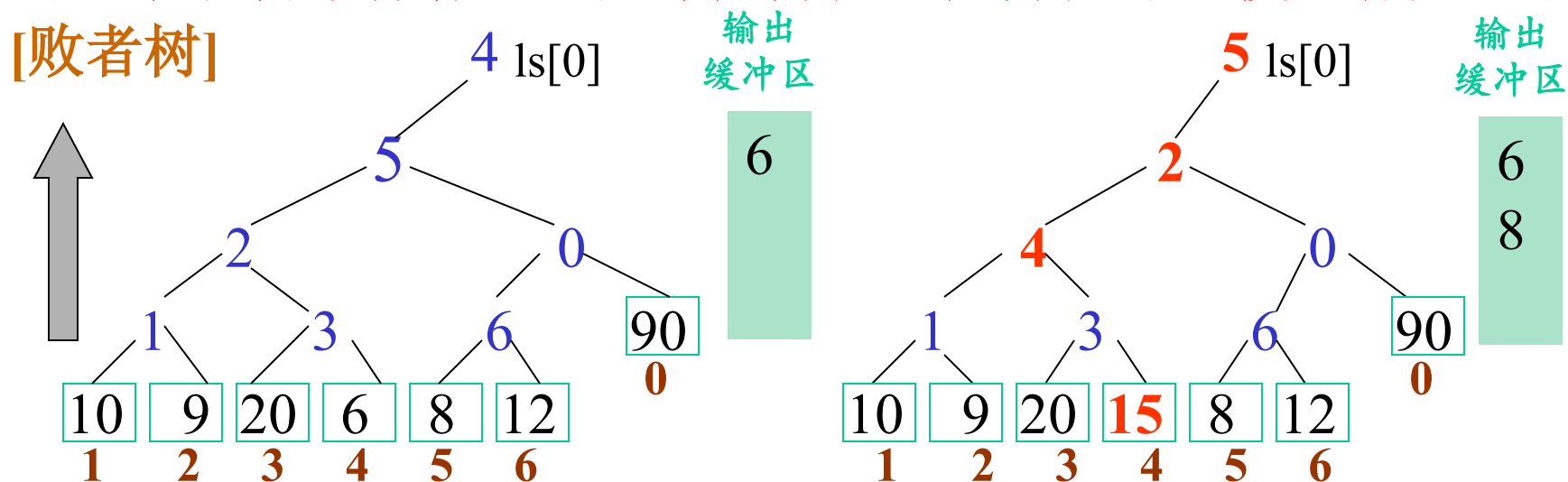


胜者树的特点：胜者留在树中，在两两比较后，在树结构中记录胜者，从叶子结点依次比到根，最后的胜者在根。



败者树的特点：在树结构中记录败者，胜者向上走，参加下一轮比赛，从叶子结点依次比到根，在根上记录最后一次的败者，胜者输出出去

实现时，败者树中保存的是多路归并段的归并段号，便于补充记录值



10	9	20	6	8	12	90
14	22	24	15	16	11	92
26	38	30	25	50	18	97
:	:	:	:	:	:	:

7路合并败者树

10	9	20	*	8	12	90
14	22	24	15	16	11	92
26	38	30	25	50	18	97
:	:	:	:	:	:	:

重构后的败者树

败者树的特点：记录败者，胜者参加下一轮比赛

败者树的优点：重构时修改结点较少。

11.4 置换-选择排序

m 个初始归并段做 k 路平衡归并排序时归并的趟数为 $\log_k m$ ；为了减少归并趟数，也可以减小 m 的值。**如何减小初始归并段的个数**（也就是增加归并段的长度）？

问题：在内存长度一定时，假设内存中可容纳 M 条记录，按照通常的内部排序方法，一次内部排序只能对 M 条记录进行排序，这样初始归并段的长度是 M ；即初始归并段长度受到内存中可容纳记录数的限制。

解决：使用**置换选择(replace selection)**算法，平均情况下可以产生可以生成两倍内存长度的初始归并段。

用于在多路归并前，产生较长的初始归并段，减少初始归并段数量。

初始归并段长度，这又受到计算机内存大小的制约，又该如何解决这个问题呢？利用置换-选择排序（replacement-selection sort）可以达到这个目的（也用败者树实现）：

置换 - 选择排序是在选择排序的基础上进行：

- 1、当内存可以进行k个记录的选择排序时，从k个记录中选出最小关键字记录（值记为minimax）；此关键字记录被输出到外存。
- 2、当内存中空出位置时，由外存中待处理的数据补充一个新的记录进来，这样有新的一组k个记录在内存中；
- 3、在这个k个记录里按如下方法寻找可以输出到外存的记录：即k个记录中大于minimax的最小值；找到后，修改minimax值为此记录key值并输出到外存，回到2。当找不到时满足条件的记录时（即内存中所有记录的关键字均比minimax小，说明内存中所有记录均不能放入已经在外存中的归并段了），此时得到了一个初始归并段（显然此归并段长度可以大于k），把这个段记录下来后再回到1继续进行。

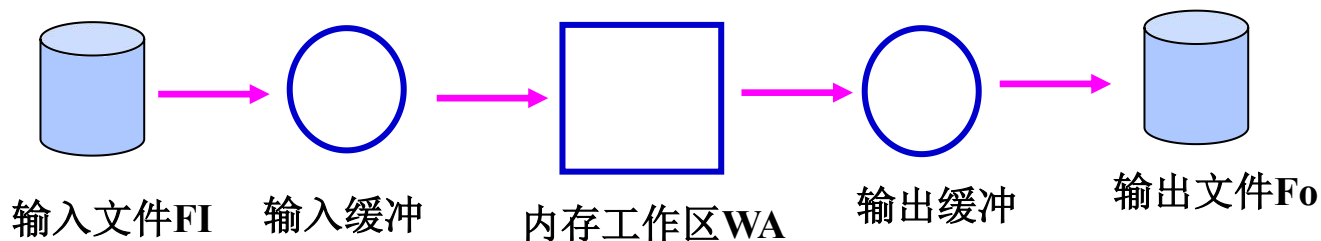
反复上述过程，直到外存中所有待处理数据均被处理完。

[示例]

FO	WA(4个记录)	FI
空	空	36,21,33,87,23,7,62,16, 54,43,29,...
空	36,21,33,87	23,7,62,16,54, 43,29,...
21	36,23,33,87	7,62,16,54, 43,29,...
21,23	36,7,33,87	62,16,54,43,29,...
21,23,33	36,7,62,87	16,54,43,29,...
21,23,33,36	16,7,62,87	54,43,29,...
21,23,33,36,62	16,7,54,87	43,29,...
21,23,33,36,62,87	16,7,54,43	29,...
21,23,33,36,62,87 7	16,29,54,43	...
...		

置换选择排序

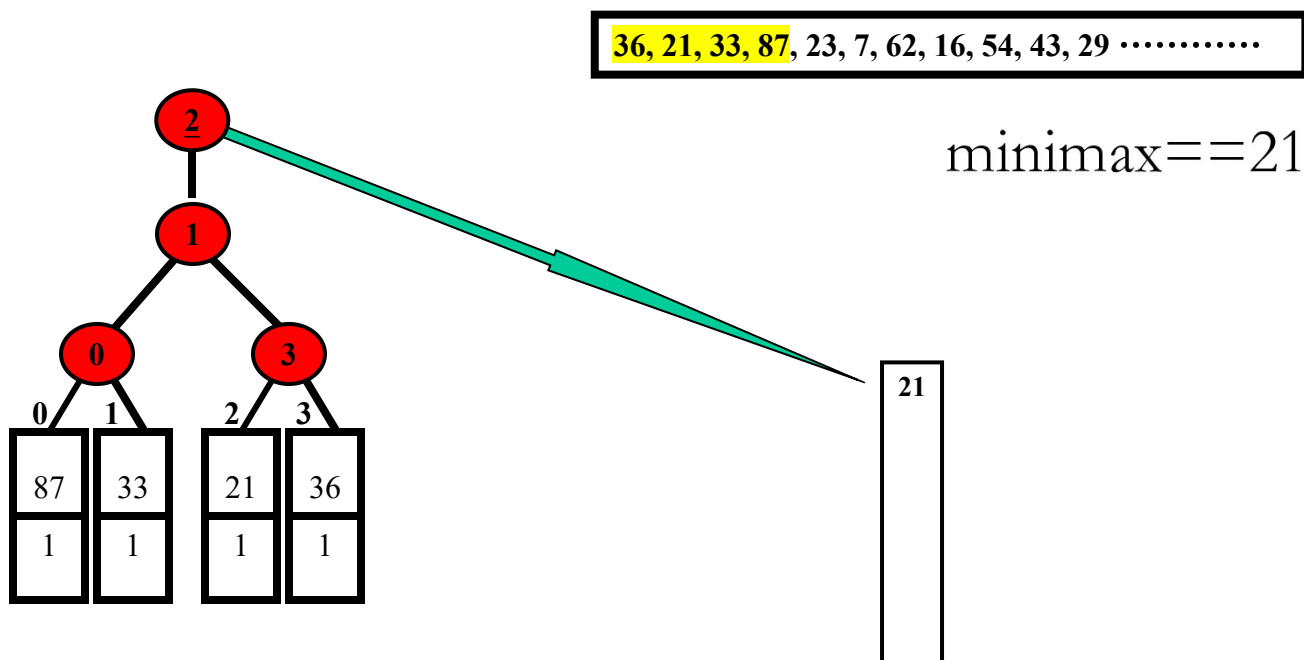
内存工作区可容纳 w 个记录



- (1) 从FI输入 w 个记录到WA;
- (2) 从WA中选择关键字最小的记录, 记为**minimax**
- (3) 将**minimax**输出到FO中;
- (4) 若FI不空, 从FI输入下一个记录到WA;
- (5) 从WA中所有关键字**比minimax**的关键字大的记录中选出最小关键字记录, 作为新的**minimax**记录。
- (6) 重复 (3) - (5) 直至WA中选不出新的**minimax**。到此得到一个初始归并段
- (7) 重复 (2) - (6), 直至WA为空。

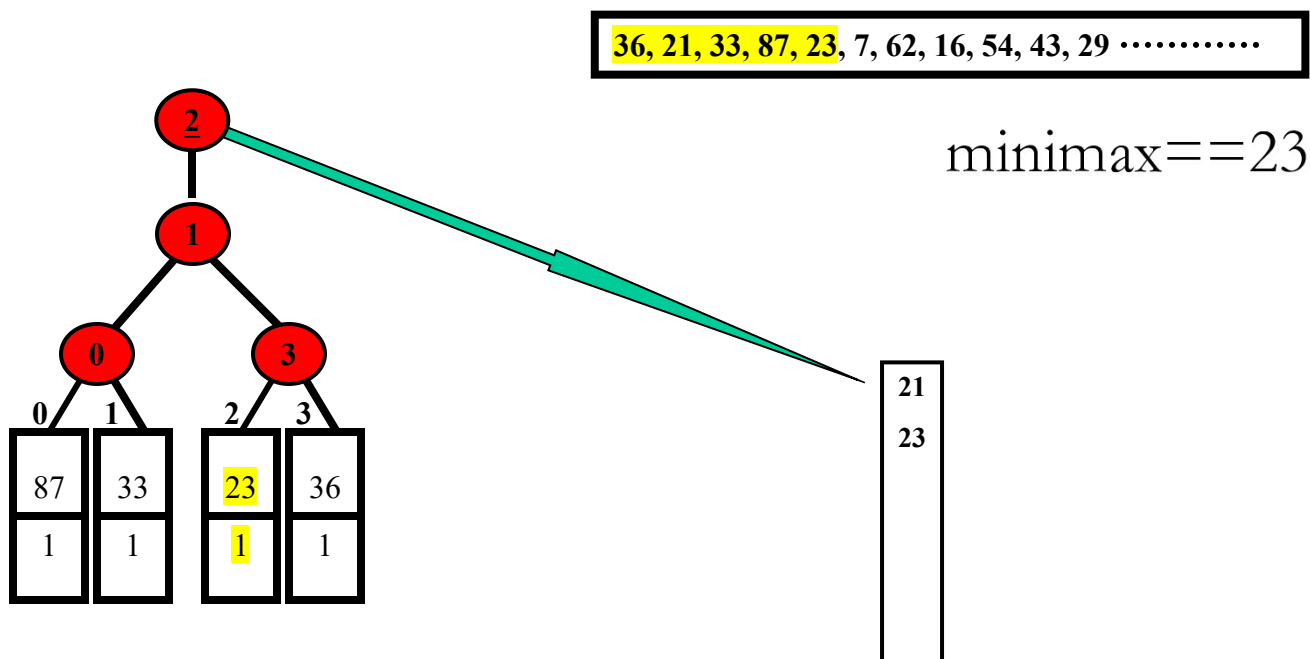
采用败者树来实现置换-选择排序：

- 1、对每个记录加入段号来记录其可能归属的归并段，初始进入败者树的WA个记录的段号为1；
- 2、当新记录FI进入败者树时，和minimax比较小时（即不会进入当前外存中的初始归并段了），新记录的段号为当前段号加一；否则为当前段号（即可以进入当前外存中的初始归并段）。
- 3、记录进行两两比较时，按“**段号不同，段大为败；段号相同，key大为败**”的比较规则进行比较并最终确定胜者，并修改minimax值为胜者的key值。



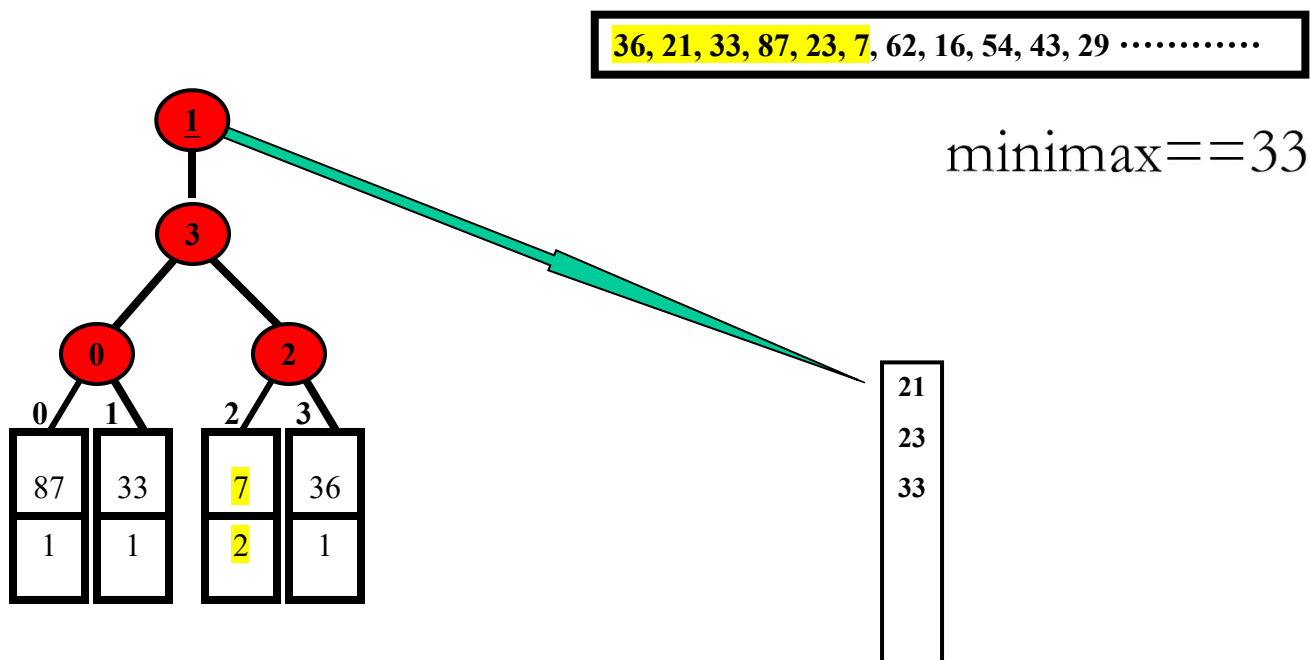
采用败者树来实现置换-选择排序：

- 1、对每个记录加入段号来记录其可能归属的归并段，初始进入败者树的WA个记录的段号为1；
- 2、当新记录FI进入败者树时，和minimax比较小时（即不会进入当前外存中的初始归并段了），新记录的段号为当前段号加一；否则为当前段号（即可以进入当前外存中的初始归并段）。
- 3、记录进行两两比较时，按“**段号不同，段大为败；段号相同，key大为败**”的比较规则进行比较并最终确定胜者，并修改minimax值为胜者的key值。



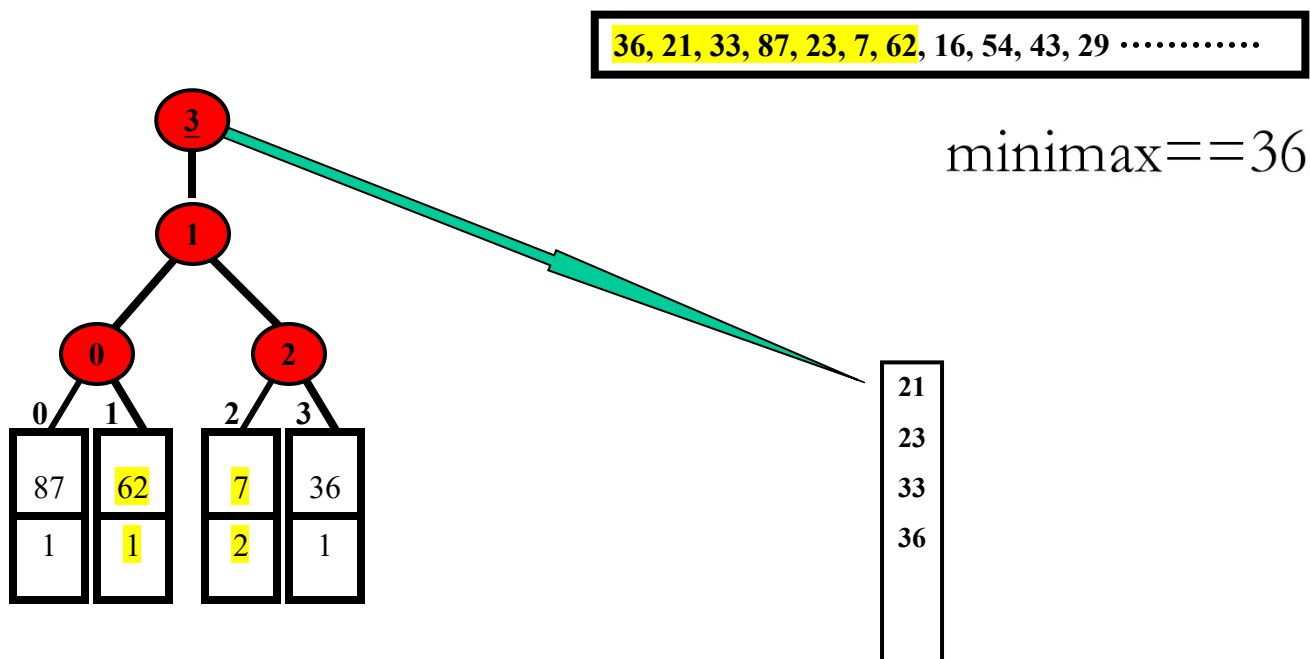
采用败者树来实现置换-选择排序：

- 1、对每个记录加入段号来记录其可能归属的归并段，初始进入败者树的WA个记录的段号为1；
- 2、当新记录FI进入败者树时，和minimax比较小时（即不会进入当前外存中的初始归并段了），新记录的段号为当前段号加一；否则为当前段号（即可以进入当前外存中的初始归并段）。
- 3、记录进行两两比较时，按“**段号不同，段大为败；段号相同，key大为败**”的比较规则进行比较并最终确定胜者，并修改minimax值为胜者的key值。



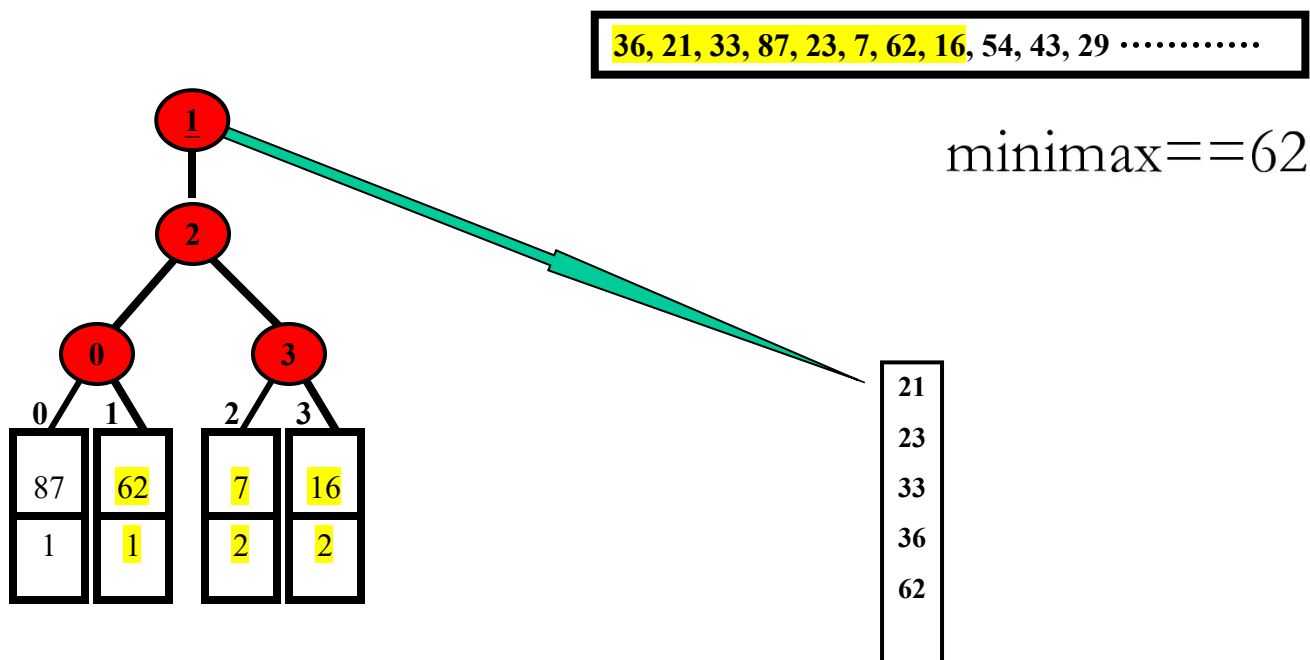
采用败者树来实现置换-选择排序：

- 1、对每个记录加入段号来记录其可能归属的归并段，初始进入败者树的WA个记录的段号为1；
- 2、当新记录FI进入败者树时，和minimax比较小时（即不会进入当前外存中的初始归并段了），新记录的段号为当前段号加一；否则为当前段号（即可以进入当前外存中的初始归并段）。
- 3、记录进行两两比较时，按“**段号不同，段大为败；段号相同，key大为败**”的比较规则进行比较并最终确定胜者，并修改minimax值为胜者的key值。



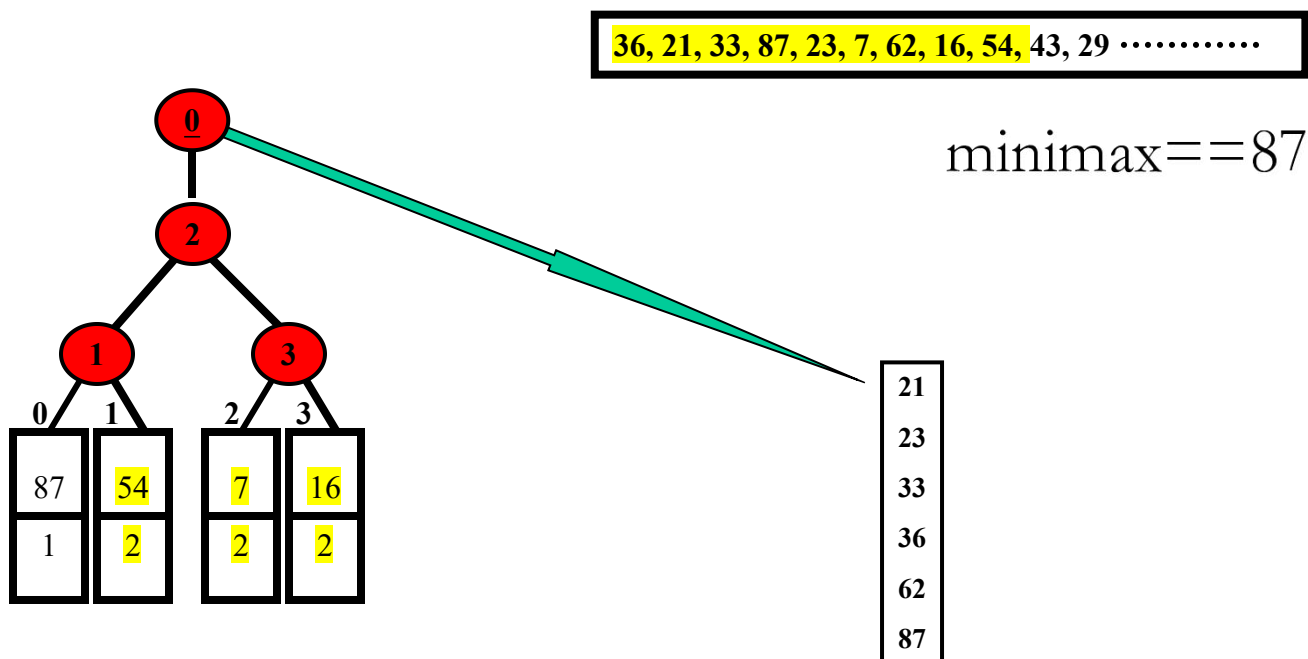
采用败者树来实现置换-选择排序：

- 1、对每个记录加入段号来记录其可能归属的归并段，初始进入败者树的WA个记录的段号为1；
- 2、当新记录FI进入败者树时，和minimax比较小时（即不会进入当前外存中的初始归并段了），新记录的段号为当前段号加一；否则为当前段号（即可以进入当前外存中的初始归并段）。
- 3、记录进行两两比较时，按“**段号不同，段大为败；段号相同，key大为败**”的比较规则进行比较并最终确定胜者，并修改minimax值为胜者的key值。



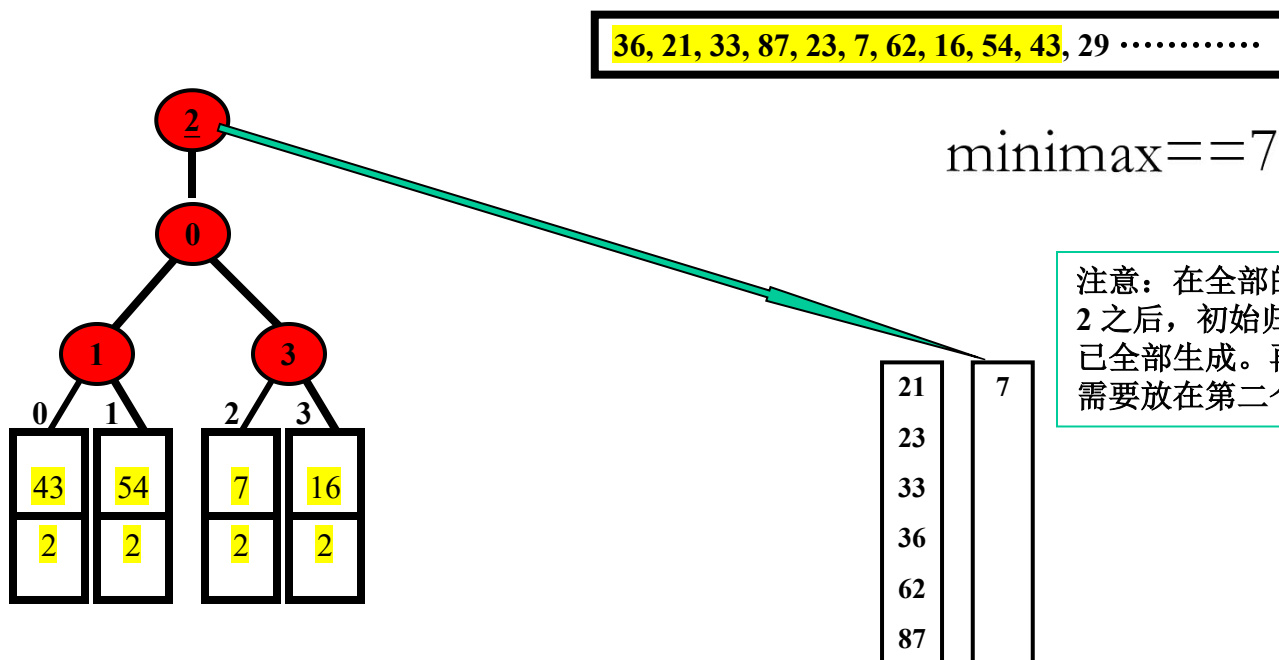
采用败者树来实现置换-选择排序：

- 1、对每个记录加入段号来记录其可能归属的归并段，初始进入败者树的WA个记录的段号为1；
- 2、当新记录FI进入败者树时，和minimax比较小时（即不会进入当前外存中的初始归并段了），新记录的段号为当前段号加一；否则为当前段号（即可以进入当前外存中的初始归并段）。
- 3、记录进行两两比较时，按“**段号不同，段大为败；段号相同，key大为败**”的比较规则进行比较并最终确定胜者，并修改minimax值为胜者的key值。



采用败者树来实现置换-选择排序：

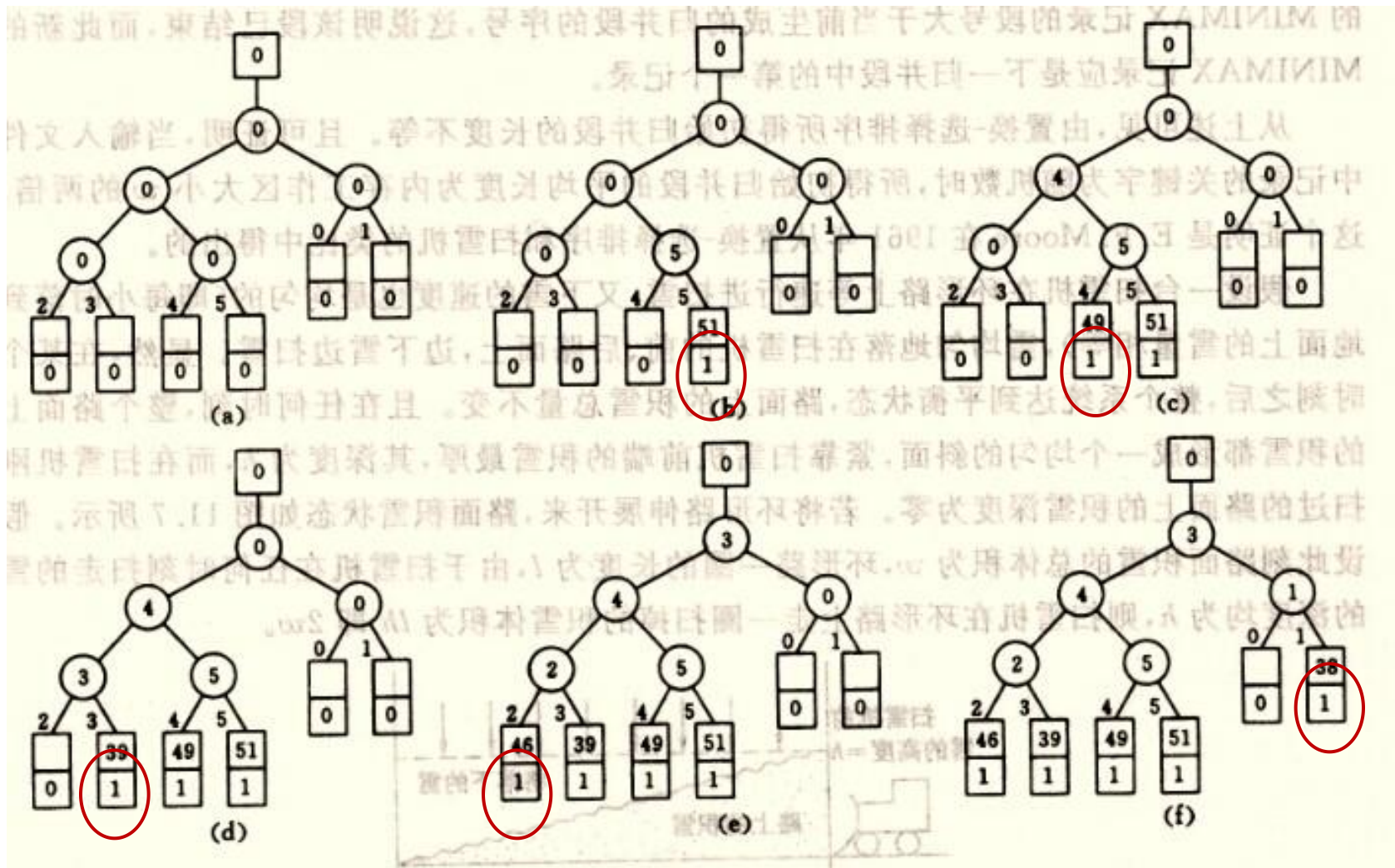
- 1、对每个记录加入段号来记录其可能归属的归并段，初始进入败者树的WA个记录的段号为1；
- 2、当新记录FI进入败者树时，和minimax比较小时（即不会进入当前外存中的初始归并段了），新记录的段号为当前段号加一；否则为当前段号（即可以进入当前外存中的初始归并段）。
- 3、记录进行两两比较时，按“**段号不同，段大为败；段号相同，key大为败**”的比较规则进行比较并最终确定胜者，并修改minimax值为胜者的key值。



注意：在全部的段号标志变成2之后，初始归并段1的记录已全部生成。再输出记录时，需要放在第二个归并段了。

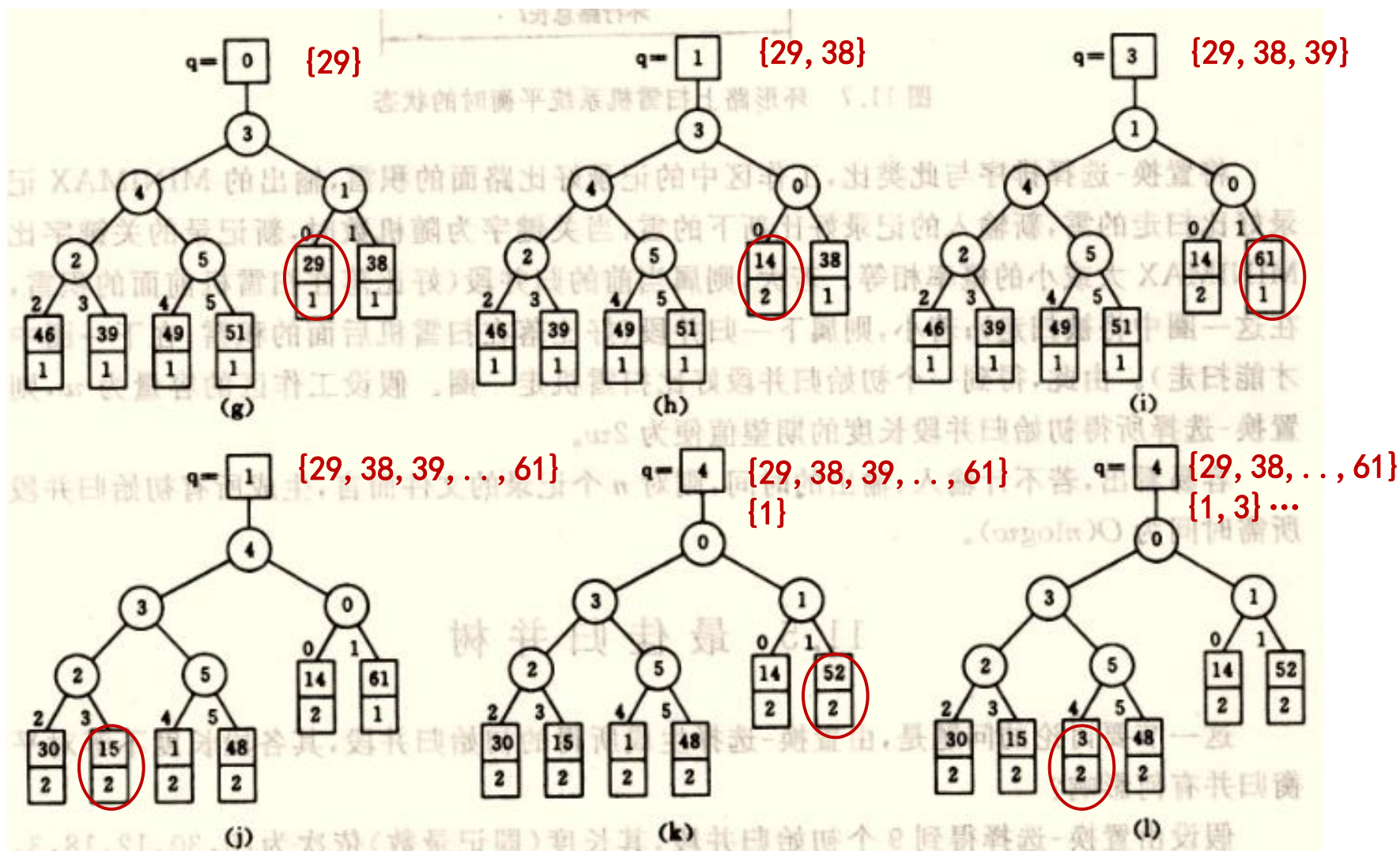
置换-选择排序可通过败者树实现 P₃₀₃图11.6 WA=6

{ 51,49,39,46,38,29,14,61,15,30,1,48,52,3,63,27,4,... }



段号不同，段大为败；段号相同，key大为败

{ 51,49,39,46,38,29,14,61,15,30,1,48,52,3,63,27,4,... }

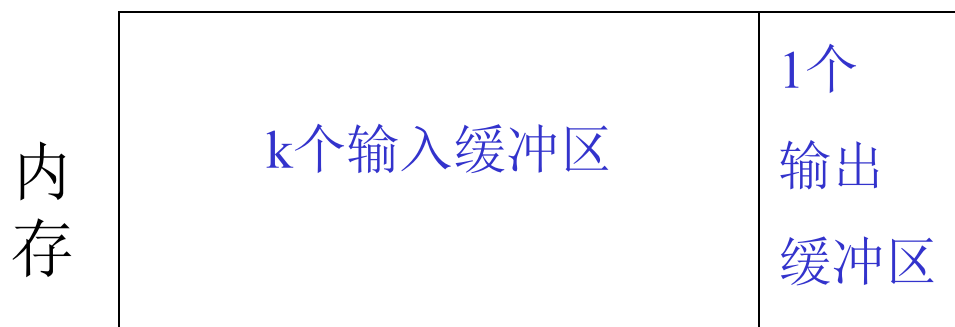


[置换-选择排序的效果]

- 所得初始排序段的长度不等
- 当输入文件记录的关键字大小随机分布时，初始归并段的长度为内存工作区大小 w 的两倍

[缓冲区及并行操作]

使输入、输出和内部归并三种操作同时进行（硬件间的并行），也能降低总的外部排序时间。但应综合考虑缓冲区数目、工作区大小 等参数的设置。



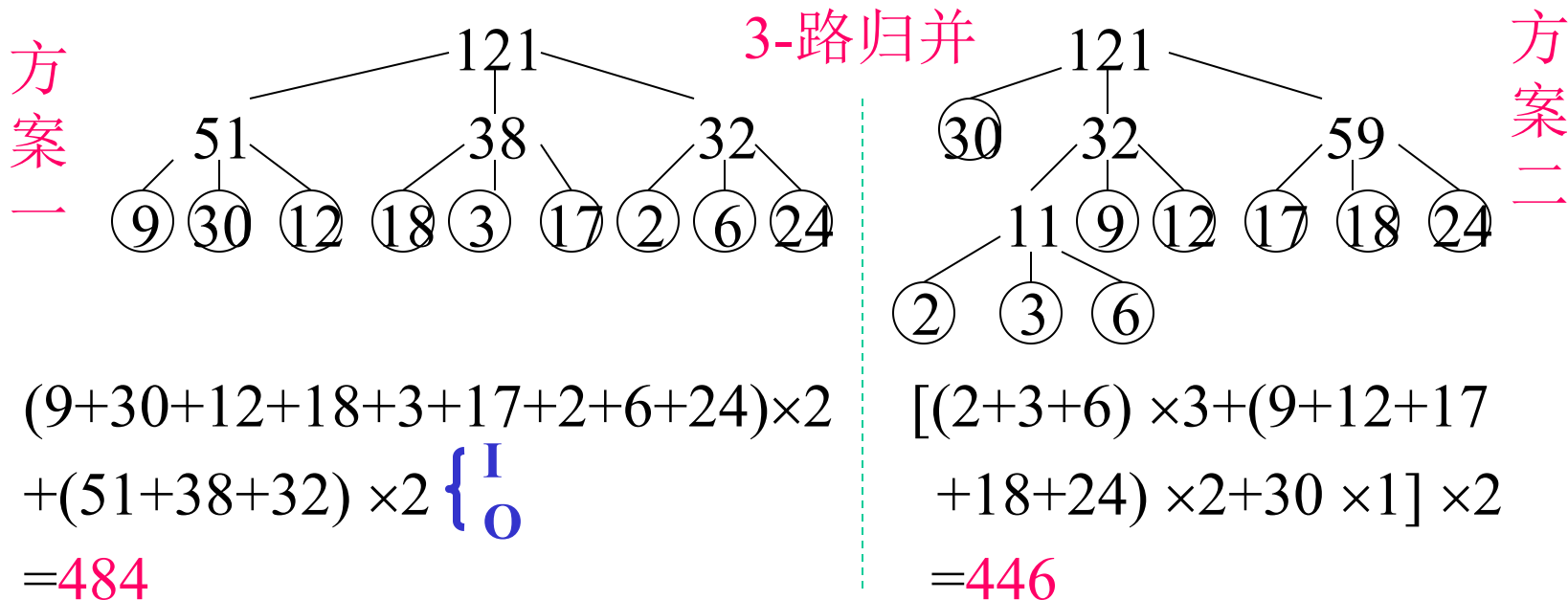
11.5 最佳归并树

[最佳归并树的引入]

进行k路归并时，初始归并段的不同搭配，会导致归并过程中**I/O的次数**不同。

设有9个初始归并段，**记录个数(长度)**分别为：

9, 30, 12, 18, 3, 17, 2, 6, 24



所有叶结点加权外通路长度的2倍

[最佳归并树的构造]

最佳归并树即k叉(阶)哈夫曼树。

设初始归并段为m个，进行k-路归并

1) 若 $(m-1) \bmod (k-1) \neq 0$

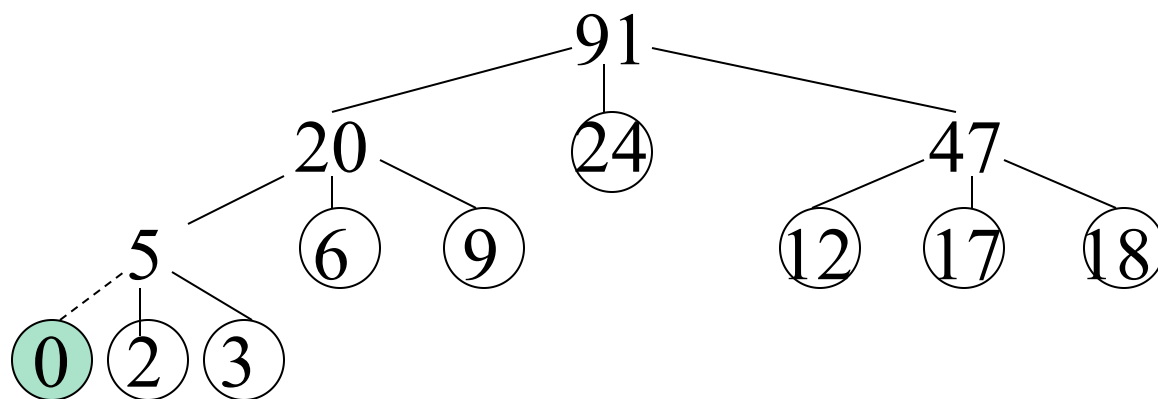
则需附加 $(k-1) - (m-1) \bmod (k-1)$ 个长度为0的虚段，
以使每次归并都可以对应k个段

2) 按照哈夫曼树的构造原则（权值越小的结点离根结点越远）构造最佳归并树，权值为归并段长度。

[示例]

长度分别为2, 3, 6, 9, 12, 17, 18, 24的8个初始归并段进行3路归并, 求最佳归并树并计算总I/O次数。

【解】 $m=8$ $k=3$, 则 $(m-1) \bmod (k-1) = 1$, 需要补充1个虚段



总I/O次数:

$$[(2+3) \times 3 + (6+9+12+17+18) \times 2 + 24 \times 1] \times 2 = 326$$

本章学习要点

- 了解外排序需要两个基本步骤
 - 生成初始归并段
 - 多趟归并
- 了解常规外排序时内外存交换次数的计算方法
- 理解提高外排序效率的两个手段是减少初始归并段个数（利用置换-选择排序）和增加归并路数（进行多路归并）
- 掌握k叉哈夫曼树的概念和最佳归并树的构造方法